

Міністерство освіти і науки України
Національний технічний університет України
“Київський політехнічний інститут імені Ігоря Сікорського”
Факультет інформатики та обчислювальної техніки
Кафедра інформаційних систем та технологій

Лабораторна робота №3

ДІАГРАМА РОЗГОРТАННЯ. ДІАГРАМА КОМПОНЕНТІВ. ДІАГРАМА ВЗАЄМОДІЙ ТА ПОСЛІДОВНОСТЕЙ.

E-mail клієнт

Виконав
студент групи ІА–22:
Артамонов Дмитро
Перевірив:
Мягкий Михайло Юрійович

Київ 2024

Зміст

Теоретичні відомості..... 4
Реалізація частини функціональної системи..... 5
Діаграма послідовностей..... 6
Діаграма розгортання..... 7
Діаграма компонентів..... 8
Висновок..... 9

Тема: Діаграма розгортання. Діаграма компнентів. Діаграма взаємодій та послідовностей.

Мета: Проаналізувати тему. Розробити діаграму розгортання, діаграму компонентів та діаграму послідовностей для проектованої системи.

Хід роботи

..15 E-mail клієнт (singleton, builder, decorator, template method, interpreter, SOA)

Поштовий клієнт повинен нагадувати функціонал поштових програм Mozilla Thunderbird, The Bat і т.д. Він повинен сприймати і коректно обробляти

---

pop3/smtp/imap протоколи, мати функції автонастройки основних поштових провайдерів для України (gmail, ukr.net, i.ua), розділяти повідомлення на папки/категорії/важливість, зберігати чернетки незавершених повідомлень, прикріплювати і обробляти прикріплені файли.

Теоретичні відомості

У лабораторній роботі 3 розглядається створення UML-діаграм для моделювання фізичного розгортання, компонування та взаємодії елементів програмної системи.

1. **Діаграма розгортання (Deployment Diagram)** відображає фізичне розміщення програмного забезпечення на апаратних вузлах. Основними елементами є вузли, що представляють фізичне обладнання або середовища виконання, які з'єднуються зв’язками для демонстрації шляхів передачі інформації.
2. **Діаграма компонентів (Component Diagram)** показує розподіл системи на окремі модулі або компоненти. Такі діаграми ілюструють зв’язки між компонентами, полегшуючи структуризацію коду та забезпечуючи його повторне використання.
3. **Діаграма послідовностей (Sequence Diagram)** моделює часову послідовність взаємодії об'єктів. Вона відображає, які об'єкти взаємодіють, і порядок передачі повідомлень між ними.

Ці діаграми допомагають отримати повне уявлення про архітектуру системи та визначити оптимальну структуру для подальшого розгортання і розвитку.

Реалізація частини функціональної системи

У процесі реалізації функціоналу системи, описаного у попередній лабораторній, було створено низку класів для роботи з основними сутностями, зокрема: Attachment, AutoConfigProvider, Confirmation, Draft, Email, EmailCategory, Folder, HttpResponse, MailAccount та User. Для взаємодії з базою даних були додані відповідні репозиторії, такі як: AttachmentRepository, AutoConfigProviderRepository, ConfirmationRepository, DraftRepository, EmailCategoryRepository, EmailRepository, FolderRepository, MailAccountRepository та UserRepository. Логіка додатка реалізована через сервіси EmailService і UserService з відповідними реалізаціями в EmailServiceImpl та UserServiceImpl. Також було створено утиліту EmailUtils для допоміжної логіки. На зображенні нижче представлена структура проєкту після реалізації цього функціоналу (рис. 1).

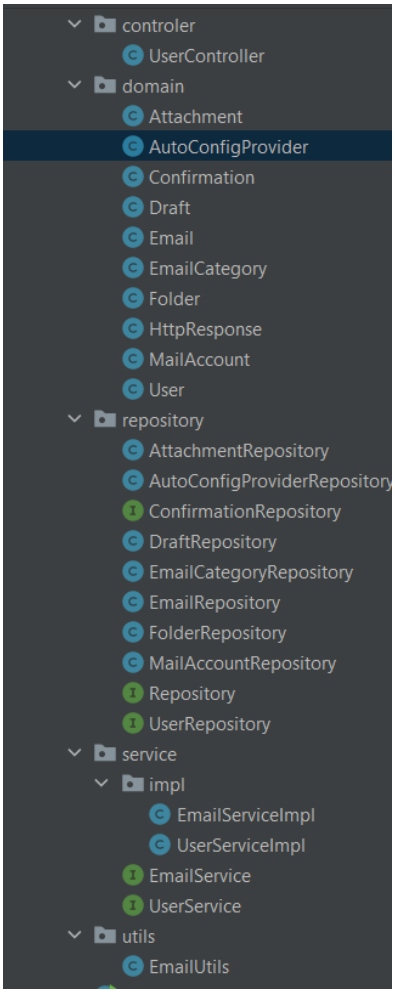
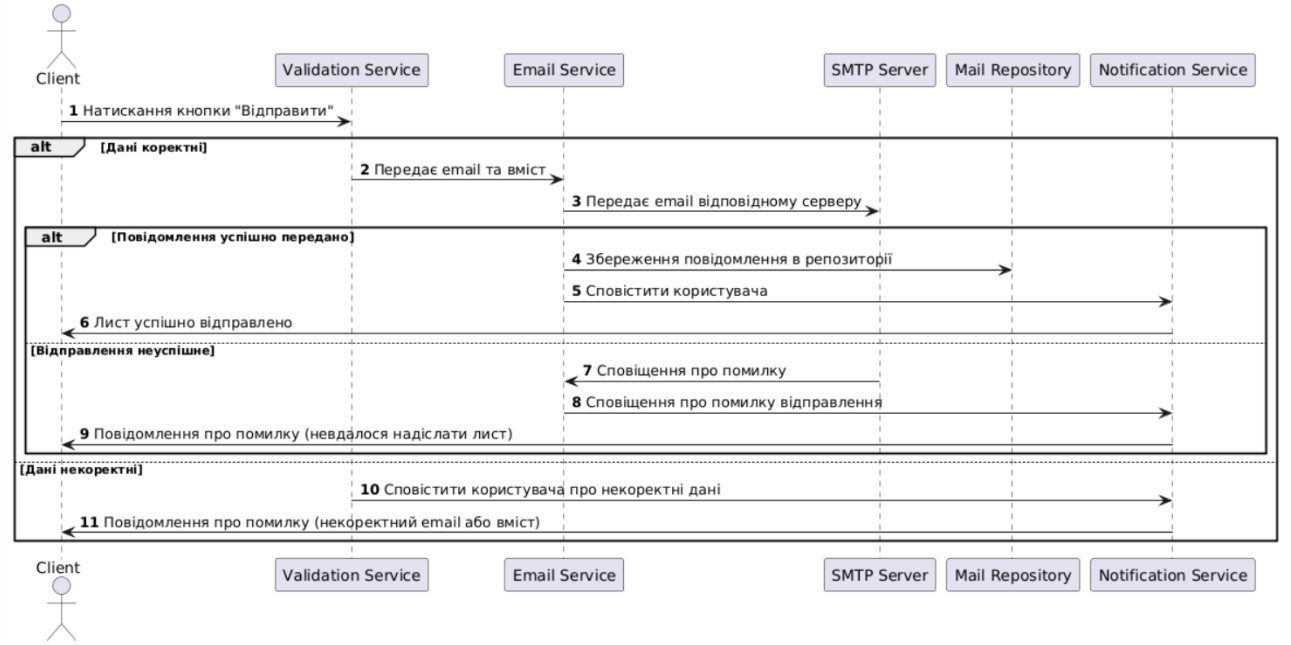


Рисунок 1.

Діаграма послідовностей

Діаграма послідовностей для процесу надсилання Email.



Опис діаграми послідовностей

1. Натискання кнопки "Відправити"

- **Клієнт** ініціює процес відправлення листа, натискаючи відповідну кнопку в інтерфейсі.

2. Перевірка даних

- **Validation Service** перевіряє правильність введених даних (адреси отримувача, вміст повідомлення).
- Якщо **дані коректні**, сервіс передає email і вміст до **Email Service**.
- Якщо на етапі перевірки даних виявлено помилки (некоректний email або вміст), **Validation Service** сповіщає клієнта про помилку і не передає лист на відправлення.

3. Передача повідомлення на SMTP сервер

- **Email Service** передає сформоване повідомлення на **SMTP-сервер**, який відповідає за надсилання листа отримувачу.

3-1. Успішне відправлення

- Якщо відправлення повідомлення через SMTP успішне, **Email Service** зберігає повідомлення в **Mail Repository.**
- Потім **Email Service** сповіщає клієнта про те, що лист було успішно відправлено.

3-2. Помилка під час відправлення

- Якщо SMTP-сервер не зміг доставити лист, **Email Service** отримує відповідне повідомлення про помилку.
- **Email Service** надсилає це сповіщення про помилку до **Notification Service**, який інформує клієнта про невдалу спробу відправлення листа.

Діаграма розгортання

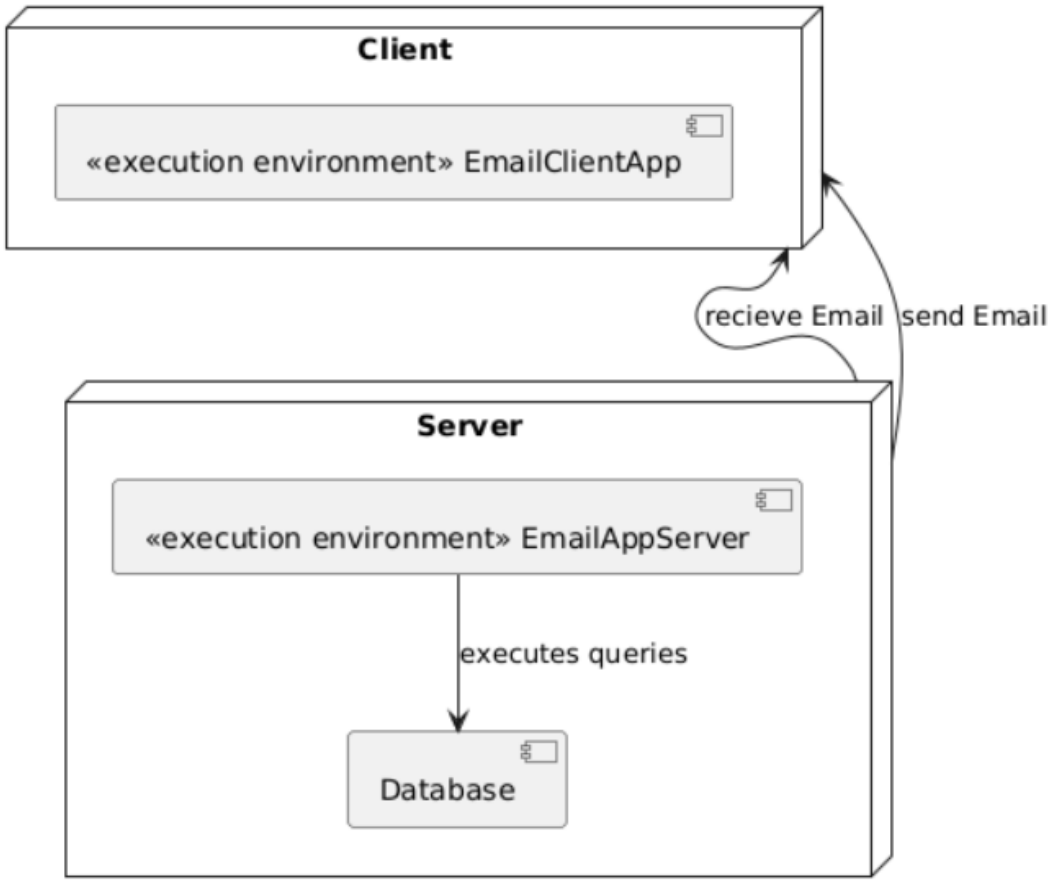


Рисунок 2.

Діаграма розгортання показує структуру системи електронної пошти, що складається з двох основних компонентів: клієнтського застосунку та серверного додатка, який відповідає за бізнес-логіку.

1. **Клієнтський пристрій (Client)** – на комп'ютері користувача запущено email клієнт (EmailClientApp). Цей додаток використовується для відправки та отримання електронних листів. Він взаємодіє із сервером через захищений протокол HTTPS.
2. **Серверний пристрій (Server)** – на сервері розгорнуто додаток (EmailAppServer), який відповідає за обробку бізнес-логіки, включаючи перевірку коректності email та подальшу пересилку на SMTP сервер. Сервер також виконує запити до бази даних, де зберігаються та обробляються дані про листи (адресати, конфігурації тощо).
3. **База даних** – серверний додаток звертається до бази даних для зберігання та отримання інформації про електронні листи та користувачів. Взаємодія з базою відбувається через запити, виконувані сервером.

Клієнтський застосунок передає запити серверному, а сервер, у свою чергу, обробляє їх, виконуючи відповідні операції з базою даних і пересилаючи листи через зовнішній SMTP сервер, або отримуючи через IMAP.

Діаграма компонентів

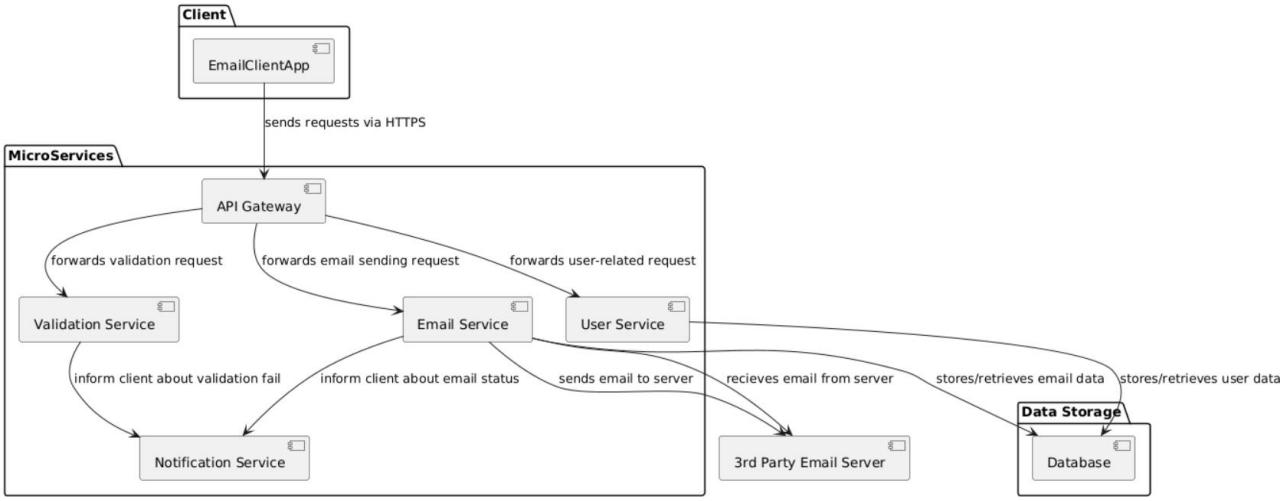


Рисунок 3.

Опис діаграми компонентів

1. **Client:**

- **EmailClientApp**: Це клієнтський застосунок, з якого користувачі взаємодіють із системою електронної пошти. Він відправляє запити до API Gateway.

2. **Microservices**:
   - **API Gateway**: Служба, яка отримує запити від клієнта та пересилає їх до відповідних мікросервісів. Вона обробляє запити на валідацію, надсилання електронних листів та запити, пов'язані з користувачами.

     Взаємодії:

     - Пересилає запити до **Validation Service** для валідації.
     - Пересилає запити до **Email Service** для надсилання електронних листів.
     - Пересилає запити до **User Service** для управління користувачами.
   - **Validation Service**: Відповідає за перевірку коректності створеного користувачем email. Якщо валідація не проходить, він повідомляє клієнта через **Notification Service**.
   - **Email Service**: Цей сервіс відповідає за обробку електронних листів. Він:
     - Взаємодіє з **3rd Party Email Server** для надсилання та отримання електронних листів.
     - Інформує **Notification Service** про статус надсилання електронної пошти (успіх або помилка).
   - **User Service**: Відповідає за управління даними користувачів, зокрема зберігання та отримання інформації про користувачів з бази даних.
3. **Notification Service**: Цей сервіс інформує користувачів про статус надсилання електронної пошти та результати валідації через API Gateway.
4. **Data Storage**: Загальне сховище для зберігання електронних листів та даних користувачів.

Посилання на репозиторій проекту: https://github.com/qjsoq/Email-App

Висновок: В данній лабораторній роботі я навчився розробляти діаграми розгортання, компонентів та діаграми взаємодій та послідовностей.